IQ-Learn Primer

Garg, Chakraborty, Cundy, Song, and Ermon (2021)

EconIRL Technical Notes

1 Problem and Motivation

IQ-Learn (Garg et al., 2021) is a model-free inverse reinforcement learning method that parameterizes the Q-function directly and recovers the reward as a byproduct. The original paper targets continuous state-action spaces where transition dynamics are unknown. In this primer we work in the finite discrete setting of DDC models, where transition probabilities $p(s'|s,a)$ are known or consistently estimated. This allows exact computation of continuation values and reveals that the IQ-Learn chi-squared objective is algebraically equivalent to a penalized conditional log-likelihood.

The main finding is that IQ-Learn combines two familiar objects. The first term is the standard multinomial logit likelihood that fits expert actions through a softmax policy. The second term penalizes the squared magnitude of the Bellman-implied reward at expert-visited states. The Q-function is the single free parameter that determines both the policy (via softmax) and the implied reward (via the inverse Bellman operator). No inner Bellman loop is needed because the smoothed value function is an explicit closed-form function of Q. The derivation requires only two algebraic identities and is verified numerically with comparisons to NFXP.

Unlike NFXP, IQ-Learn does not enforce the Bellman equation as a hard constraint. Instead, the chi-squared regularizer penalizes the implied reward magnitude, selecting from the equivalence class of observationally equivalent rewards by favoring solutions with small implied reward. Unlike GLADIUS (Kang et al., 2025), IQ-Learn does not explicitly enforce Bellman consistency, which means the recovered Q may not satisfy the Bellman equation and the implied reward may not be a valid structural primitive. GLADIUS addresses this by adding a Bellman error term to the objective and proving global convergence via the Polyak-Łojasiewicz condition.

2 Model and Notation

A finite-state, finite-action MDP with state space $\mathcal{S}$, action set $\mathcal{A}(s)$, transitions $p(s'|s,a)$, reward $r(s,a)$, discount $\beta \in (0,1)$, and T1EV scale $\sigma > 0$. The key objects are as follows.

Table 1: MDP objects.

Symbol	Name	Definition
$Q(s,a)$	Choice-specific value	$r(s,a) + \beta \sum_{s'} p(s'\|s,a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s,a)/\sigma)$
$\pi(a\|s)$	Policy (CCP)	$\exp(Q(s,a)/\sigma) / \sum_{a'} \exp(Q(s,a')/\sigma)$
$\mathrm{td}(s,a)$	Temporal difference	$Q(s,a) - \beta \sum_{s'} p(s'\|s,a)\, V(s')$
α	Regularization	Controls penalty strength; $1/(4\alpha\sigma)$ is the penalty weight

The temporal difference $\mathrm{td}(s,a)$ is the reward implied by a given Q-function through the inverse Bellman operator. At the Bellman fixed point, $\mathrm{td}(s,a) = r(s,a)$. For a general Q that does not satisfy the Bellman equation, $\mathrm{td}(s,a)$ is the reward that would rationalize Q as a fixed point. The IQ-Learn objective is built from two algebraic identities that hold for any Q, not only for Q^*.

3 Core Theory: Penalized MLE Equivalence

Proposition 1 (Advantage equals scaled log CCP)**.** *For any $Q(s,a)$, define $V(s)$ and $\pi(a|s)$ from the table above. Then*

$$Q(s,a) - V(s) = \sigma \log \pi(a|s). \tag{1}$$

Proof. Take the log of the softmax: $\log \pi(a|s) = Q(s,a)/\sigma - \log(\sum_{a'} \exp(Q(s,a')/\sigma))$. Multiply by σ. □

Proposition 2 (Temporal difference equals implied reward)**.** *If Q satisfies the Bellman fixed point $Q = \Lambda_\sigma(Q)$, then $\mathrm{td}(s,a) = r(s,a)$.*

Proof. At the fixed point, $Q(s,a) = r(s,a) + \beta \sum_{s'} V(s')\, p(s'|s,a)$. Subtract $\beta \sum_{s'} V(s')\, p(s'|s,a)$ from both sides. □

The IQ-Learn chi-squared objective. Given expert observations $(s_i, a_i)_{i=1}^N$, IQ-Learn minimizes over Q:

$$\mathcal{L}(Q) = -\frac{1}{N}\sum_{i=1}^{N}\big[Q(s_i,a_i) - V(s_i)\big] + \frac{1}{4\alpha N}\sum_{i=1}^{N}\big[\mathrm{td}(s_i,a_i)\big]^2. \tag{2}$$

Step 1. By Proposition 1, the first term equals $-(\sigma/N)\sum_i \log \pi(a_i|s_i) = -(\sigma/N)\,\mathrm{LL}(Q)$, the scaled negative conditional log-likelihood.

Step 2. Writing $\hat{r}_i \equiv \mathrm{td}(s_i,a_i)$, the second term is $(4\alpha N)^{-1}\sum_i \hat{r}_i^2$, the average squared implied reward.

Step 3. Combining and multiplying by $-N/\sigma$ converts minimization of $\mathcal{L}$ into maximization of:

$$\boxed{\mathrm{LL}(Q) \;-\; \frac{1}{4\alpha\sigma}\sum_{i=1}^{N}\hat{r}_i^2} \tag{3}$$

where $\hat{r}_i = Q(s_i,a_i) - \beta \sum_{s'} V(s')\, p(s'|s_i,a_i)$ and $V(s) = \sigma \log \sum_a \exp(Q(s,a)/\sigma)$.

Expression (3) is a penalized conditional log-likelihood. The first term is the multinomial logit log-likelihood with Q-values as utilities. The second term is an ℓ^2 penalty on the Bellman-implied reward, weighted by the expert occupancy. The penalty is quadratic in $\hat{r}$ but not in Q (because V depends on Q through the logsumexp). No inner Bellman solve appears because V is an explicit function of Q.

Identification. Multiple reward functions rationalize the same policy. For any potential $h : \mathcal{S} \to \mathbb{R}$, the shaped reward $r'(s,a) = r(s,a) + h(s) - \beta \sum_{s'} h(s')\, p(s'|s,a)$ yields the same optimal policy (Ng et al., 1999). IQ-Learn does not resolve this non-identification. The chi-squared regularizer selects the element with smallest implied reward at expert-visited states. This has no economic interpretation, unlike the anchor normalizations of AAIRL (Lee et al., 2026) which set the exit payoff to zero.

Comparison to GLADIUS. Both IQ-Learn and GLADIUS combine a log-likelihood term with a Bellman-consistency penalty. The key difference is how the continuation value $\mathbb{E}[V(s')|s,a]$ is handled. IQ-Learn with known transitions computes it exactly. GLADIUS works model-free and introduces a dual variable $\zeta(s,a) \approx \mathbb{E}[V(s')|s,a]$ to correct the double-sampling bias. GLADIUS also explicitly enforces Bellman consistency and has global convergence guarantees via the PL condition, while IQ-Learn does not enforce or guarantee Bellman satisfaction.

4 Estimation Algorithm

Algorithm 1: IQ-Learn (tabular DDC setting)

Input: Expert data $\{(s_i, a_i)\}_{i=1}^N$, transitions $p(s'|s,a)$, discount β, scale σ, regularization α

Initialize $Q(s,a)$ for all (s,a) (e.g., zeros or from behavioral cloning)

repeat

`// Forward pass: compute` V `and` π `from current` Q

$V(s) \leftarrow \sigma \log \sum_a \exp(Q(s,a)/\sigma)$ for all s

$\pi(a|s) \leftarrow \exp(Q(s,a)/\sigma) / \sum_{a'} \exp(Q(s,a')/\sigma)$ for all (s,a)

`// Implied reward at expert observations`

$\hat{r}_i \leftarrow Q(s_i, a_i) - \beta \sum_{s'} p(s'|s_i, a_i)\, V(s')$ for $i = 1, \ldots, N$

`// Penalized log-likelihood`

$\mathrm{LL} \leftarrow \sum_{i=1}^N \log \pi(a_i|s_i)$

$\mathcal{J}(Q) \leftarrow \mathrm{LL} - \frac{1}{4\alpha\sigma} \sum_{i=1}^N \hat{r}_i^2$ `// Eq. (3)`

`// Update` Q `by gradient ascent`

$Q \leftarrow Q + \eta\, \nabla_Q \mathcal{J}(Q)$ `// or L-BFGS-B`

until *convergence*

`// Extract structural objects`

$\hat{r}(s,a) \leftarrow Q(s,a) - \beta \sum_{s'} p(s'|s,a)\, V(s')$ for all (s,a)

$\hat{\pi}(a|s) \leftarrow \mathrm{softmax}(Q(s,\cdot)/\sigma)$

Output: $\hat{Q}$, $\hat{r}$, $\hat{\pi}$

Implementation notes. In the tabular setting with known transitions, the gradient $\nabla_Q \mathcal{J}$ is computed by automatic differentiation through the logsumexp. No inner Bellman loop is needed. The EconIRL implementation uses L-BFGS-B for the outer optimization. In continuous spaces where p is unknown, the original IQ-Learn paper replaces the exact continuation value with sampled next states and adds a separate policy network, but these are implementation details for the continuous case. The regularization parameter α controls how tightly the Bellman equation is enforced: $\alpha \to 0$ forces $\hat{r} \to 0$ at expert states, $\alpha \to \infty$ removes the penalty and reduces to unconstrained logit with no structural content.

5 Results

Table 2: IQ-Learn vs NFXP on Rust bus (90 bins, $\beta = 0.9999$). IQ-Learn trades log-likelihood for smaller implied reward.

	True	IQ-Learn	NFXP
Op. cost ($\times 10^{-3}$)	1.000	9.643	1.231
Repl. cost	3.000	1.539	3.011
Log-likelihood	—	-4292.5	-4263.2
Time (sec)	—	3.1	12.4

On the 2-state, 2-action MDP with $\beta = 0.9$ and $\sigma = 0.5$, IQ-Learn with $\alpha = 1$ recovers reward differences close to truth but achieves a lower log-likelihood than NFXP (-567.7 vs -563.4 with 1000 observations). The gap reflects the cost of regularization: NFXP maximizes fit alone, while IQ-Learn

trades fit for smaller implied reward ($\sum \hat{r}_i^2 = 0.25$ vs 0.76 for NFXP). Both recover policies close to the truth.

The reward shaping invariance property confirms that the regularizer breaks the equivalence class. Under a potential $h = (2, -1)$, the shaped reward yields an identical policy but the sum of squared implied rewards rises from 1.93 to 11.00. The penalty in Eq. (3) selects the Q-function with smaller implied reward, effectively choosing a normalization without economic content. This contrasts with AAIRL's anchors (exit payoff = 0, absorbing state value = 0), which have transparent economic meaning.

IQ-Learn is suited for quick exploratory estimation where the Bellman equation need not hold exactly and where the researcher wants a single-pass estimator without inner loops. For structural counterfactuals requiring Bellman-consistent rewards, NFXP or GLADIUS is preferred.

6 EconIRL Implementation

The estimator is in `src/econirl/estimation/iq_learn.py` as `IQLearnEstimator`. Run `python papers/primers/iq_learn_example.py` to regenerate results.

Table 3: Code mapping.

Step	Method	Operation
Forward pass	`_compute_values()`	$V = \sigma \,\mathrm{logsumexp}(Q/\sigma)$
Implied reward	`_compute_td()`	$\hat{r} = Q - \beta\, P\, V$
Penalized LL	`_objective()`	$\mathrm{LL} - (4\alpha\sigma)^{-1} \sum \hat{r}^2$
Optimize	`minimize_lbfgsb()`	L-BFGS-B over Q-table

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.iq_learn import IQLearnEstimator, IQLearnConfig

env = RustBusEnvironment(operating_cost=0.001, replacement_cost=3.0,
                         num_mileage_bins=90, discount_factor=0.9999)
panel = env.generate_panel(n_individuals=200, n_periods=100, seed=42)

iq = IQLearnEstimator(config=IQLearnConfig(alpha=1.0))
result = iq.estimate(panel=panel, utility=LinearReward.from_environment(env)
    ,
                     problem=env.problem_spec, transitions=env.
                        transition_matrices)
```

Table 4: IQ-Learn in context.

	NFXP	CCP	IQ-Learn	GLADIUS	AAIRL
Bellman enforced	✓	✓		✓	
Transition-free				✓	✓
Inner loop	✓				
Global convergence	✓	✓		✓	
Identification	parametric	parametric	regularizer	anchor	anchor

Comparison across estimators. **References.**

Garg, D., Chakraborty, S., Cundy, C., Song, J., and Ermon, S. (2021). IQ-Learn: Inverse soft-Q learning for imitation. *NeurIPS 35*.

Kang, E.H., Yoganarasimhan, H., and Jain, L. (2025). An empirical risk minimization approach for offline inverse RL and DDC. arXiv:2502.14131.

Lee, P.S., Sudhir, K., and Wang, T. (2026). Modeling serialized content consumption: Adversarial IRL for DDC. Yale SOM.

Ng, A., Harada, D., and Russell, S. (1999). Policy invariance under reward transformations. *ICML*.

Rust, J. (1987). Optimal replacement of GMC bus engines. *Econometrica* 55(5):999–1033.